CONTRACT NOTE REWORK

PDF / Template Management

Estimate 1 - Technical Walkthrough

~13.5 developer days (9.0 required + testing)

D55 Consulting

July 2026

BrytEnergy

Background

Contract notes are the PDF documents BRYT Energy produces when a customer signs up to an energy contract. Today they are generated by a fixed pipeline: an XML file lands in S3, a .NET SVG engine renders it to HTML, and that HTML is converted to a PDF. Any change to the layout, wording, or terms means a developer has to edit and redeploy code.

That makes the current system slow to change and entirely developer-dependent. Business users cannot adjust a template, add a new contract type, or update Terms & Conditions without raising a development request.

Estimate 1 replaces the rendering mechanism with a self-service template management system inside the existing BrytAdminPortal. Business users compose templates visually, define the rules that pick the right template automatically, and manage reusable content like headers, footers, and T&Cs, all without developer involvement. The S3-triggered entry point stays the same, so upstream systems are unaffected.

> ### What changes, in one line
>
> The trigger and the output stay the same (XML in, PDF out). What changes is the middle: a hard-coded .NET renderer becomes a business-managed library of templates and sections, rendered by pdf-me and stitched with pdf-lib.

How it works

The system has two halves. The first is a management experience in the Admin Portal where users build and maintain templates. The second is an automated render pipeline that runs when contract data arrives and produces the finished PDF.

A template is an ordered set of sections. Each section is a small pdf-me layout (positioned text, multi-variable text, and tables). Sections render independently and are then stitched into one document. Common content, headers, footers, and Terms & Conditions, lives as shared sections that can be reused across many templates.

When contract data arrives, the pipeline decides which template to use by evaluating each template's selection rule in priority order and taking the first match. It then renders that template's sections and stitches them into the final PDF.

The moving parts

Five components make up the solution. The first three are user-facing; the last two run automatically.

COMPONENT	WHERE IT LIVES	WHAT IT DOES
Template management UI	Angular module in BrytAdminPortal	CRUD for templates, sections, and shared sections; drag-to-reorder priority and composition
Rules engine UI	Angular module in BrytAdminPortal	Visual tree editor for the selection rule that picks a template automatically
Section editor	Embedded pdf-me Designer (React web component)	Visual field placement on a base PDF, opened in a modal
Render pipeline	Single AWS Lambda	Evaluates rules, renders each section with @pdfme/generator, stitches with pdf-lib
Storage	DynamoDB + S3	DynamoDB holds template/section metadata and rules; S3 holds the schema JSON and output PDFs

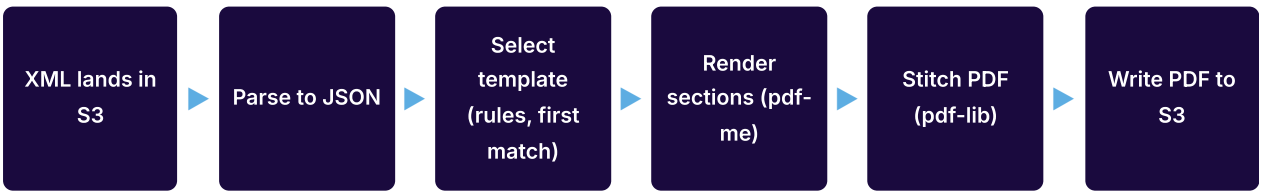
System architecture

The Admin Portal talks to a set of Lambda APIs for templates, sections, and rules, backed by DynamoDB (metadata) and S3 (schema JSON). The render pipeline is triggered by an XML drop in S3, reads the same metadata and schema, and writes the finished PDF to the output bucket.

Admin Portal (top) manages content; APIs persist it; the render pipeline consumes the same storage.

Render pipeline flow

When an XML file lands in the input bucket, a single Lambda runs the whole flow end to end: parse the data, pick the template by evaluating rules in priority order, render each section, stitch them into one PDF, and write the result out.



A failure at any stage halts the run and logs an error record; no partial PDF is written.

Key design decisions

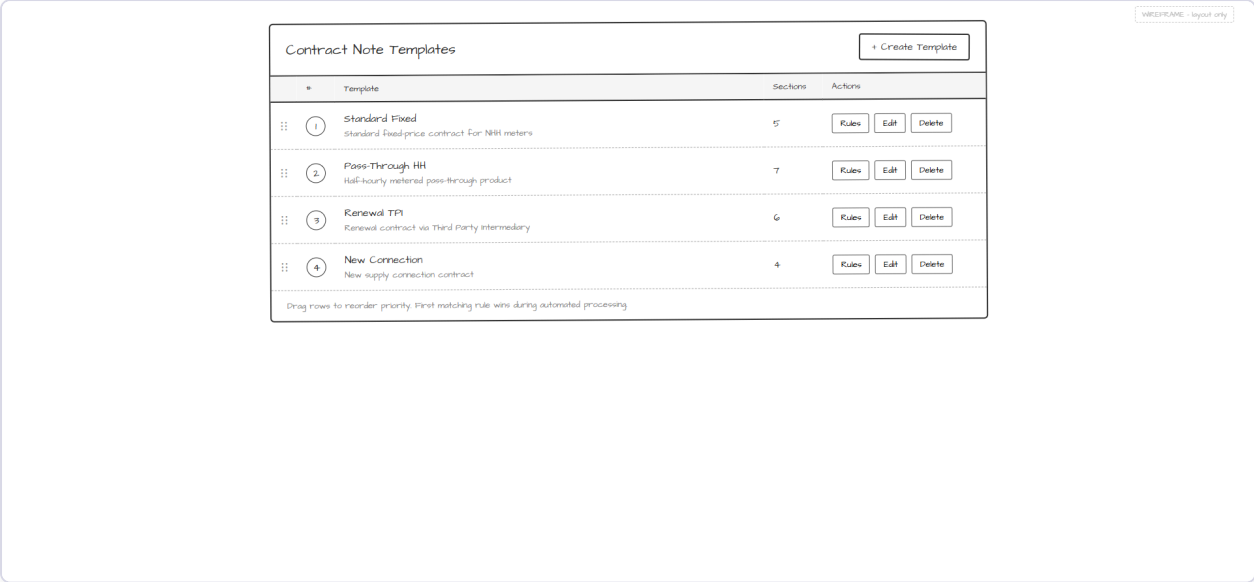
The design settled on a handful of choices that shape the rest of the build.

DECISION	CHOICE	WHY
Templates	Ordered list of sections	Sections render independently, so dynamic content (like multi-page tables) is handled cleanly, then stitched
Shared content	Reusable shared sections	Headers, footers, and T&Cs are defined once and referenced, not copied. Edit once, applies everywhere
T&Cs	Just shared sections, positioned last	No special upload mechanism; T&Cs use the same editor and are always placed at the end
Selection rules	Specification pattern (rule tree)	AND/OR/NOT logic with EQUALS/IN/LESS_THAN/MORE_THAN leaves; first match by priority wins
Editor embedding	pdf-me Designer as a web component	Wraps the React designer as a custom element, embedded in Angular via a modal; avoids a bridge library
Storage	DynamoDB single-table + S3	Fast ordered queries for metadata; S3 is cost-effective for the larger schema JSON blobs
Pipeline	Single Lambda (not Step Functions)	Render-and-stitch is a synchronous flow; one Lambda avoids step-function coordination overhead

Screen-by-screen walkthrough

Six screens make up the template management experience. Each is shown below with the user interactions it supports and the data model working behind it.

1. Template List



The entry point. An ordered table of every configured template. The order is meaningful: it is the priority in which selection rules are evaluated at render time.

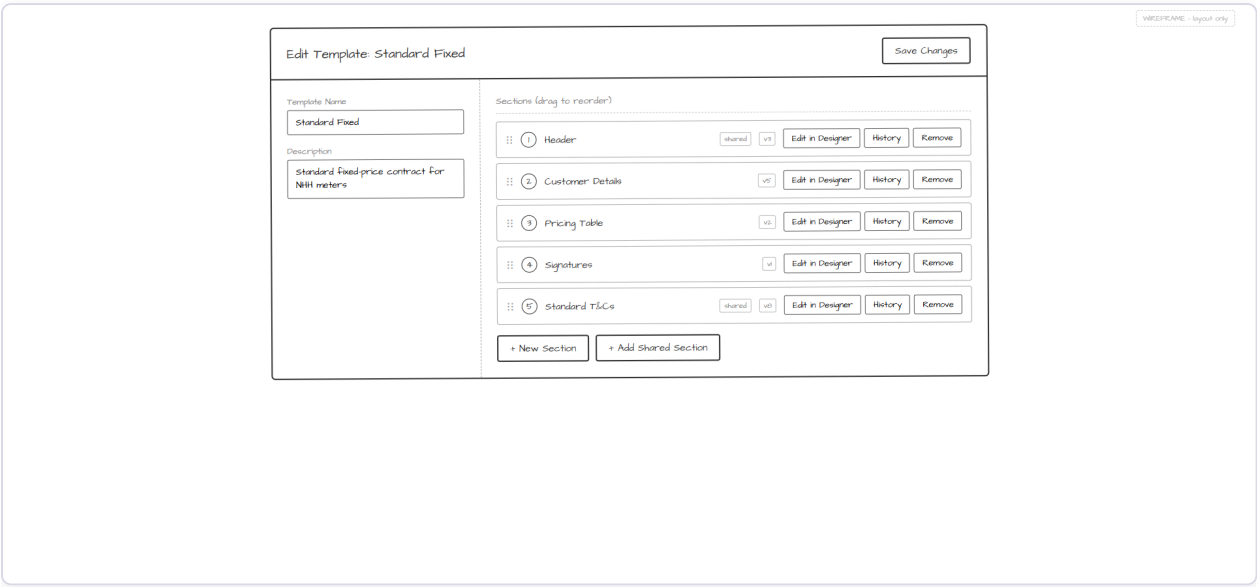
KEY INTERACTIONS

- Drag the handle to reorder priority (top = evaluated first)
- Rules button opens the rule tree editor for that template
- Edit opens the template composition screen
- Delete prompts for confirmation before removing
- Create Template adds a new template at the lowest priority

BEHIND THE SCREEN

- Each row is a Template record in DynamoDB (name, description, sectionCount, priority)
- Ordering is a query on the PriorityIndex GSI (all templates, sorted by priority)
- Reordering rewrites priority values to stay contiguous (1..N)
- Deleting re-numbers the remaining templates to keep priorities contiguous

2. Template Edit



Where a template's metadata and section composition are managed. The centre panel is the ordered section list; the right panel adds new or shared sections; the bottom panel is the change log.

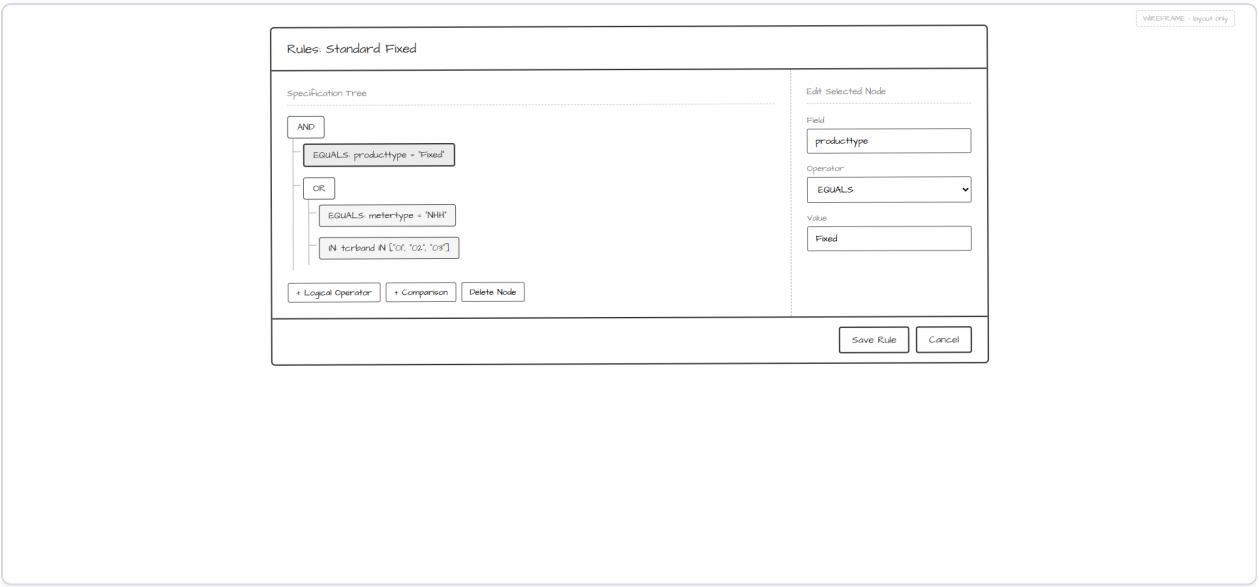
KEY INTERACTIONS

- Drag to reorder sections within the template
- Edit in Designer opens the pdf-me section editor
- History opens the version history for that section
- Add Shared Section picks from the shared library; T&Cs auto-position at the end
- A version badge (e.g. v3) shows each section's current version

BEHIND THE SCREEN

- Sections are Section records keyed by sortOrder within the template
- Shared sections are stored as references (sharedSectionId), never duplicated
- Every change writes a Template Change Log record (who, what, when)
- Adding a section appends it at max sortOrder + 1

3. Rules Configuration



The visual editor for a template's selection rule. The rule is a specification tree: logical operators (AND, OR, NOT) combining comparison leaves (EQUALS, IN, LESS_THAN, MORE_THAN).

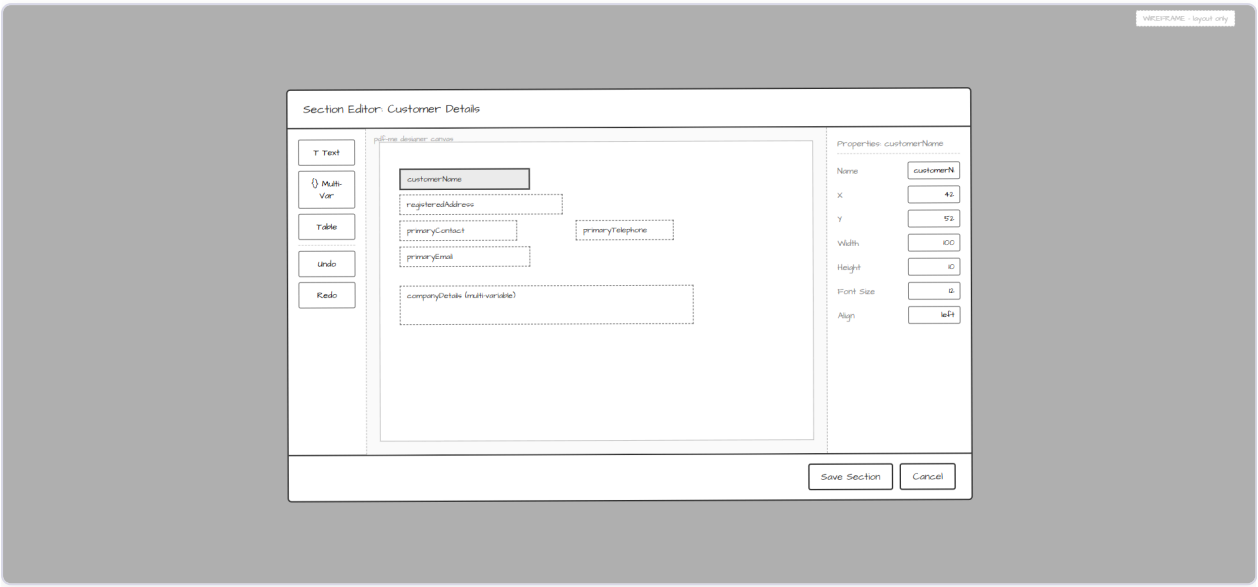
KEY INTERACTIONS

- Click a node to select it and edit it in the right panel
- Add logical operators or comparisons as children of the selected node
- Delete removes a node and its children
- Save validates the tree is well-formed before persisting

BEHIND THE SCREEN

- Serialised as a JSON tree on the template's Rule record
- AND/OR nodes have leftOperand + rightOperand; NOT has a single operand
- Leaves reference a data field (e.g. offer.producttype) and a value or value set
- Validation rejects incomplete nodes and reports their location

4. Section Editor (pdf-me Designer)



The pdf-me visual designer, embedded in a modal. Fields are positioned on a base PDF background, giving users a true what-you-see layout experience.

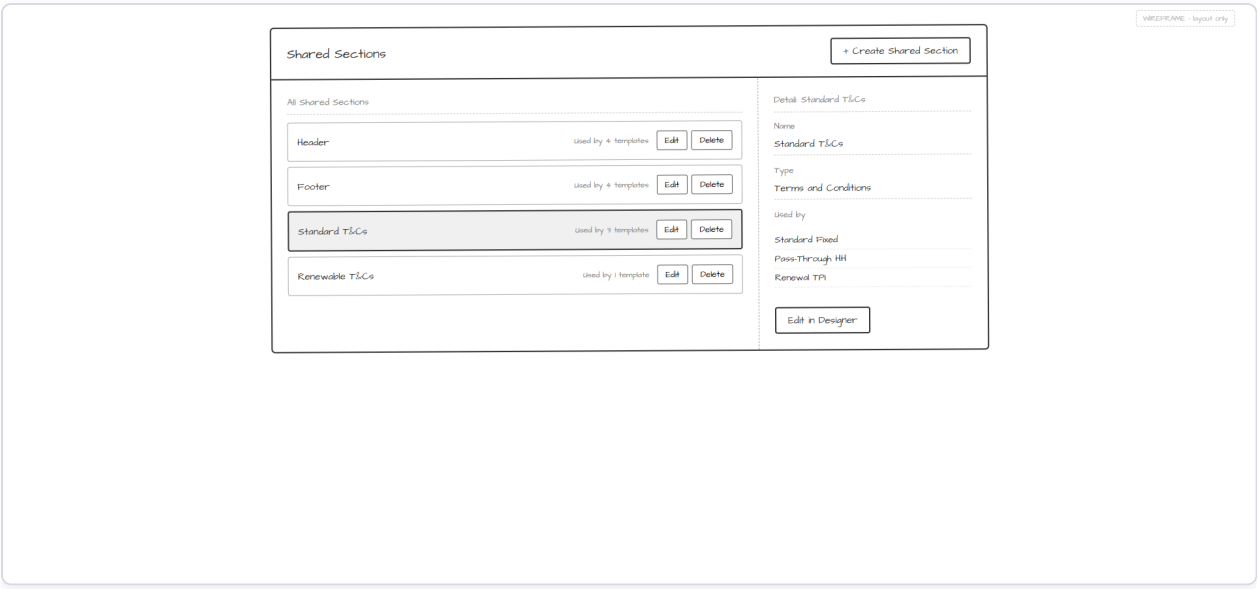
KEY INTERACTIONS

- Drag fields on the canvas to reposition them
- Click a field to edit its properties (position, size, font, alignment)
- Add text, multi-variable text, or table fields from the toolbar
- Undo/redo; Save writes the layout back

BEHIND THE SCREEN

- Each section's layout is a pdf-me Schema JSON blob stored in S3
- The schemas array holds one entry per page of the section
- Saving creates a new version rather than overwriting (see screen 6)
- The same editor is used for regular, shared, and T&C sections

5. Shared Sections Library



Manages reusable sections, headers, footers, and Terms & Conditions. Each shows how many templates use it, and the detail panel lists exactly which ones.

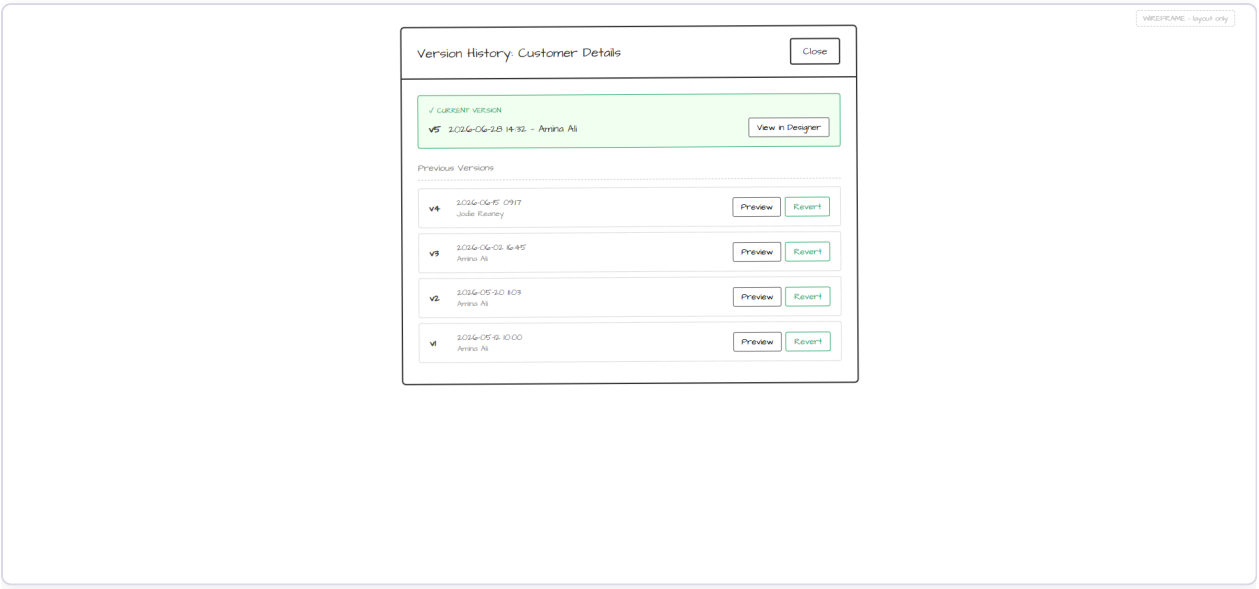
KEY INTERACTIONS

- Select a shared section to see its detail and where it is used
- Edit in Designer opens the same pdf-me editor
- Delete warns (and lists templates) if the section is still referenced
- Create prompts for a name and type (standard or T&C)

BEHIND THE SCREEN

- Shared Section records are keyed separately from templates
- A Reference record links each shared section to the templates using it
- Editing a shared section propagates to every template that references it
- Deletion is blocked while references exist

6. Section Version History



Every save of a section creates a new version, so nothing is ever lost. This screen lists the versions and lets a user preview or revert to an earlier one.

KEY INTERACTIONS

- The current version is highlighted at the top
- View in Designer opens the current version read-only
- Preview opens a historical version read-only
- Revert creates a new version from the selected one (non-destructive)

BEHIND THE SCREEN

- Each save writes a Section Version record (timestamp, user, schema location)
- Reverting copies an old version forward as a new version; intermediate versions remain
- All versions are retained indefinitely (no automatic purging)

The rules engine, in plain terms

The rules engine decides which template to use for a given contract without anyone choosing manually. Each template carries a rule; at render time the pipeline evaluates rules in the priority order from the template list and uses the first one that matches.

A rule is a tree. Branches are logical operators (AND, OR, NOT) and leaves are comparisons against fields in the contract data. For example: producttype EQUALS 'Fixed' AND (region IN [North, South] OR mpancount MORE_THAN 5).

- EQUALS - the field exactly matches a value
- IN - the field is one of a set of values
- LESS_THAN / MORE_THAN - numeric comparison against a threshold
- AND / OR / NOT - combine or negate the above

> ### First match wins
>
> Because rules are evaluated in priority order and the first match is taken, template ordering is a real business lever. More specific templates sit higher; a general fallback sits at the bottom. If nothing matches, the pipeline logs an error rather than guessing.

Data model at a glance

Everything lives in a single DynamoDB table using a PK/SK pattern, with the larger layout blobs in S3.

RECORD	HOLDS	NOTES
Template	Name, description, priority, section count	One per template; ordered via a priority GSI
Section	Name, sort order, shared reference, schema location	Owned by a template; ordered by sortOrder
Shared Section	Name, T&C flag, schema location	Reusable; referenced by many templates
Reference	Template-to-shared-section link	Tracks which templates use a shared section

RECORD	HOLDS	NOTES
Section Version	Timestamp, user, schema location	One per save; enables history and revert
Rule	Specification tree JSON	One per template; drives selection
Change Log	Change type, description, user, time	Audit trail of template changes
Schema JSON (S3)	pdf-me field layout per section	Referenced by sections and versions

Delivery breakdown

Estimate 1 is ~9.0 required days plus optional testing, ~13.5 days in total. Work is grouped as follows.

AREA	SCOPE
Infrastructure & types	DynamoDB table + GSI, S3 buckets, API Gateway routes, shared TypeScript types, rule validation
Template API	List, create, get, update, delete, reorder handlers with priority management
Section API	Section CRUD, schema get/save, version history + revert, shared section CRUD, change log
Rules API	Get and save the specification tree, with validation
Render pipeline	Rule evaluator, template selection, section renderer, PDF stitcher, XML-to-JSON + S3 trigger
Angular frontend	Template list, template edit, rules config, section editor web component, shared sections, version history
Integration	CDK wiring, IAM, S3 event trigger, portal navigation, end-to-end validation

Testing note

The build is designed around 30 correctness properties (ordering, round-trips, rule evaluation, PDF page-count preservation, and so on). Property-based and integration tests are marked optional in the plan and can be deferred for a faster MVP, but they are recommended given the pipeline's role in producing legally significant documents.